REPORTE DE PRÁCTICA NO. 1

Análisis de Expresiones Regulares

ALUMNO: Ángel Ortega Gómez
Dr. Eduardo Cornejo-Velázquez

1. Introducción

El análisis de cadenas de símbolos es un proceso fundamental dentro de la informática teórica y práctica, ya que permite identificar y validar patrones que forman parte de un lenguaje formal. En esta práctica se aborda el estudio y la aplicación de técnicas para el reconocimiento de palabras a partir de un alfabeto definido, apoyándose en los conceptos de gramáticas formales y lenguajes regulares como base teórica para el análisis de símbolos.

La práctica se enfoca en la identificación y clasificación de distintos tipos de cadenas, tales como números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos, utilizando dos enfoques principales.

- El primero consiste en el desarrollo de programas que emplean estructuras de datos y algoritmos de análisis para recorrer y evaluar carácter por carácter las cadenas de entrada.
- El segundo enfoque se basa en el uso de expresiones regulares, las cuales permiten describir de manera compacta y precisa los patrones que definen a cada tipo de palabra dentro de un lenguaje regular.

Se busca comprender la relación entre la teoría de los lenguajes formales y su aplicación práctica en lenguajes de programación de alto nivel. De esta manera, la práctica permite reforzar la importancia de un proceso sistemático y metodológico para el análisis de cadenas de símbolos.

2. Marco teórico

Lenguajes formales y gramáticas

Los lenguajes formales son una herramienta fundamental en la teoría de la computación, ya que permiten describir de manera precisa conjuntos de cadenas formadas a partir de un alfabeto finito. Estos lenguajes se definen mediante reglas formales que establecen qué combinaciones de símbolos son válidas, lo cual resulta esencial para el diseño y análisis de lenguajes de programación, compiladores e intérpretes [1].

Una gramática formal es un modelo matemático utilizado para generar lenguajes formales y se define como una tupla compuesta por símbolos terminales, símbolos no terminales, un símbolo inicial y un conjunto de producciones. A través de estas producciones es posible derivar cadenas de símbolos que pertenecen al lenguaje definido por la gramática. Este enfoque proporciona una base rigurosa para el análisis sintáctico y la validación de cadenas dentro de sistemas computacionales [2].

Gramáticas regulares y lenguajes regulares

Las gramáticas regulares, también conocidas como gramáticas de tipo 3 dentro de la jerarquía de Chomsky, se caracterizan por generar lenguajes regulares. Este tipo de gramáticas posee una estructura sencilla en la que las producciones siguen patrones lineales, lo que permite que los lenguajes generados puedan ser reconocidos por autómatas finitos [3].

Los lenguajes regulares son ampliamente utilizados en aplicaciones prácticas debido a su facilidad de implementación y eficiencia en el reconocimiento de patrones. Son especialmente relevantes en el análisis léxico, donde se requiere identificar tokens como números, identificadores y símbolos dentro del código fuente de un programa [4].

Expresiones regulares

Las expresiones regulares constituyen una notación formal que permite describir lenguajes regulares de manera compacta y precisa. Mediante el uso de operadores como la concatenación, la unión y la cerradura de Kleene, es posible definir patrones complejos a partir de reglas simples, facilitando el análisis de cadenas de símbolos [5].

En los lenguajes de programación de alto nivel, las expresiones regulares se utilizan ampliamente para la validación de datos, el procesamiento de texto y el análisis léxico. Su uso permite reducir la complejidad del código y mejorar la legibilidad al encapsular las reglas de validación dentro de una única expresión formal [6].

Análisis de cadenas de símbolos

El análisis de cadenas de símbolos consiste en examinar secuencias de caracteres pertenecientes a un alfabeto con el objetivo de identificar patrones específicos y clasificarlos de acuerdo con reglas previamente definidas. Este proceso puede llevarse a cabo mediante algoritmos que recorren carácter por carácter la cadena, apoyándose en estructuras de datos que facilitan la validación de cada símbolo [7].

Este tipo de análisis es esencial en diversas áreas de la informática, particularmente en la etapa de análisis léxico de los compiladores, donde se identifican elementos como números enteros, palabras reservadas, iden-

tificadores y símbolos especiales. Un análisis correcto garantiza que las cadenas procesadas cumplan con las reglas del lenguaje y puedan ser interpretadas adecuadamente por el sistema [4].

Aplicación del análisis léxico en lenguajes de programación

El análisis léxico representa la primera fase del proceso de compilación y tiene como finalidad transformar una secuencia de caracteres en una secuencia de tokens significativos. Para ello, se emplean técnicas basadas tanto en gramáticas regulares como en expresiones regulares, las cuales permiten definir formalmente los patrones asociados a cada tipo de token [8].

La correcta implementación del análisis léxico facilita la detección de errores tempranos y contribuye a la robustez del software. Además, el uso combinado de algoritmos de análisis y expresiones regulares permite comprender la relación entre la teoría de los lenguajes formales y su aplicación práctica en el desarrollo de programas de alto nivel [1].

3. Obejtivos

Objetivo general

Desarrollar programas en un lenguaje de programación de alto nivel que permitan el análisis de cadenas de símbolos pertenecientes a un lenguaje regular, mediante la implementación de estructuras de datos, algoritmos de análisis y expresiones regulares, con el fin de identificar, clasificar y contabilizar distintos tipos de palabras.

Objetivos específicos

- Identificar el proceso básico de análisis de cadenas de símbolos a partir de un alfabeto definido.
- Implementar estructuras de datos y algoritmos para el análisis y validación de cadenas de símbolos en un lenguaje de programación de alto nivel.
- Definir el alfabeto y las expresiones regulares necesarias para reconocer números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos.
- Implementar expresiones regulares como una estrategia alternativa para el análisis, clasificación y conteo de cadenas de símbolos.
- Comparar los resultados obtenidos mediante el análisis algorítmico y el uso de expresiones regulares.

4. Herramientas empleadas

Para el desarrollo de la práctica se utilizaron diversas herramientas de software que permitieron la implementación, prueba y validación de los programas orientados al análisis de cadenas de símbolos. Dichas herramientas facilitaron tanto la codificación como la verificación del correcto funcionamiento de los algoritmos y expresiones regulares empleados.

1. Java.Lenguaje de programación de alto nivel, orientado a objetos y multiplataforma, utilizado para la implementación de los algoritmos de análisis de cadenas y el uso de expresiones regulares. Java ofrece bibliotecas estándar robustas para el manejo de texto y validación de patrones, lo que lo hace adecuado para el desarrollo de aplicaciones de análisis léxico.

2. Apache NetBeans IDE 17. Entorno de desarrollo integrado utilizado para la escritura, compilación y ejecución de los programas desarrollados en Java. NetBeans proporciona herramientas de depuración y organización del código que facilitan el desarrollo estructurado y la detección de errores durante la implementación.

3. Java Development Kit (JDK) 17. Conjunto de herramientas necesarias para el desarrollo de aplicaciones en Java, el cual incluye el compilador, la máquina virtual y las bibliotecas estándar del lenguaje. Esta versión garantiza compatibilidad con las características modernas del lenguaje utilizadas en la práctica.

4. Regex101. Herramienta en línea empleada como apoyo para el diseño, prueba y validación de expresiones regulares. Esta plataforma permite visualizar el funcionamiento de las expresiones, facilitando la corrección y optimización de los patrones utilizados para el análisis de cadenas.

5. Stack Overflow. Plataforma de consulta técnica utilizada como referencia para la resolución de dudas específicas relacionadas con el análisis de cadenas en Java. En particular, se consultó el recurso disponible en `http://stackoverflow.com/questions/12446074/how-do-i-check-every-value-in-a-java-string`, el cual sirvió como apoyo para comprender el manejo y validación de caracteres dentro de cadenas.

5. Desarrollo

5.1 Programa de análisis de cadenas con estructuras de datos

En la primera etapa de la práctica se desarrolló un programa en Java cuyo objetivo es analizar cadenas de caracteres utilizando estructuras de datos y algoritmos básicos, sin hacer uso de expresiones regulares. El programa inicia mostrando al usuario un menú de opciones mediante una estructura *switch*, en el cual se selecciona el tipo de análisis a realizar: números enteros, palabras en minúsculas, palabras en mayúsculas o identificadores.

Una vez seleccionada la opción correspondiente, el programa solicita al usuario el ingreso de una cadena de caracteres a través de un cuadro de diálogo implementado con *JOptionPane*. Al confirmar la entrada, el sistema procede a evaluar la cadena introducida y muestra el resultado del análisis mediante otro cuadro de diálogo, indicando si la palabra cumple o no con las características del tipo seleccionado.

El funcionamiento interno del programa se basa en el análisis carácter por carácter de la cadena ingresada. Para ello, se recorre la palabra mediante ciclos y se comparan los valores de cada carácter con los rangos ASCII correspondientes, verificando si se encuentran dentro del intervalo permitido para cada categoría. Por ejemplo, en el caso de las palabras en minúsculas, se valida que cada carácter se encuentre dentro del rango comprendido entre 'a' y 'z', descartando la cadena si se detecta algún símbolo fuera de dicho rango.

Para la validación de identificadores, el programa utiliza funciones propias del lenguaje Java, tales como *isJavaIdentifierStart* e *isJavaIdentifierPart*, las cuales permiten verificar si una cadena cumple con las reglas sintácticas para nombres de variables. Adicionalmente, se realiza una comparación con un arreglo que contiene las palabras reservadas del lenguaje Java, con el fin de evitar que estas sean reconocidas incorrectamente como identificadores válidos.

Este enfoque permite comprender el proceso básico de análisis de cadenas de símbolos mediante el uso de estructuras de control, arreglos y funciones propias del lenguaje, reforzando el entendimiento del análisis léxico desde una perspectiva algorítmica antes de emplear expresiones regulares como una alternativa más compacta.

Adicionalmente, el código fuente se encuentra disponible en el repositorio de GitHub: Detector de Cadenas de Caracteres

5.2 Definición del alfabeto y expresiones regulares

Para el análisis de las cadenas de símbolos se definió un alfabeto compuesto por letras del alfabeto inglés, dígitos decimales y símbolos especiales comúnmente utilizados en lenguajes de programación. Este alfabeto sirve como base para la construcción de lenguajes regulares que permiten clasificar las distintas categorías de palabras analizadas durante la práctica.

El alfabeto considerado se define como:

$$\Sigma = \{a, \ldots, z,\ A, \ldots, Z,\ 0, \ldots, 9,\ _,\ +,\ -,\ *,\ /,\ \%,\ <,\ >,\ =,\ !,\ \&,\ |,\ \hat{}\ \sim,\ (,),\ \{\},\ [],\ ;,\ ,,\ .,\ :,\ ?,\ _,\ \#,\ @\}$$

A partir de este alfabeto se establecieron las expresiones regulares necesarias para identificar cada tipo de cadena, las cuales describen formalmente los patrones que deben cumplir las palabras para pertenecer a una categoría específica.

Los números enteros se definen como secuencias formadas únicamente por dígitos decimales, por lo que su

expresión regular se establece como:

$$[0-9]+$$

Este patrón permite reconocer cualquier número entero positivo compuesto por uno o más dígitos.

Las palabras en minúsculas se definen como cadenas formadas exclusivamente por letras del alfabeto en minúscula. La expresión regular correspondiente es:

$$[a-z]+$$

Dicha expresión garantiza que la cadena contenga únicamente letras minúsculas sin la presencia de números o símbolos especiales.

De manera similar, las palabras en mayúsculas se identifican mediante cadenas compuestas solo por letras en mayúscula. La expresión regular utilizada para este caso es:

$$[A-Z]+$$

Este patrón descarta cualquier carácter que no pertenezca al rango de letras mayúsculas.

Los identificadores, utilizados comúnmente como nombres de variables en lenguajes de programación, deben iniciar con una letra o un guion bajo, seguido de una combinación de letras, dígitos o guiones bajos. La expresión regular que describe este tipo de cadenas se define como:

$$[a-zA-Z_\$][a-zA-Z0-9_\$]*$$

Este patrón asegura que la cadena cumpla con las reglas sintácticas básicas de un identificador válido.

Finalmente, los símbolos se definen como caracteres especiales que no pertenecen a las categorías anteriores y que suelen emplearse en la sintaxis de los lenguajes de programación. Estos se identifican mediante la expresión regular:

$$[+-*/\% <>=!\&|(){}[];,.:?_\#@]+$$

La cual permite reconocer diversos símbolos utilizados de forma común en la escritura de expresiones y estructuras de control.

5.3 Programa con validación mediante expresiones regulares

En esta tercera parte de la práctica se desarrolló un programa que mantiene la estructura general del programa de la sección 5.1; sin embargo, el método de análisis de las cadenas de símbolos se modifica para utilizar expresiones regulares como estrategia principal de validación. El propósito de esta sección es aplicar expresiones regulares para evaluar de manera completa las cadenas ingresadas por el usuario, aprovechando las herramientas que ofrece el lenguaje de programación Java.

El programa inicia mostrando un menú de opciones implementado mediante una estructura *switch*, en el cual el usuario selecciona el tipo de palabra que desea analizar. Posteriormente, se solicita el ingreso de una cadena de caracteres a través de un cuadro de diálogo utilizando la clase *JOptionPane*. Una vez que el usuario ingresa la cadena, el sistema procede a evaluarla comparándola directamente con la expresión regular correspondiente a la opción seleccionada.

La validación se realiza mediante el método *matches()*, el cual permite comprobar si la cadena completa cumple con el patrón definido por la expresión regular. Este enfoque difiere del análisis carácter por carácter implementado en el primer programa, ya que en este caso se evalúa la cadena de forma global, lo que reduce la complejidad del código y mejora la claridad del proceso de validación.

Para el caso específico de los identificadores, además de validar la estructura general mediante expresiones regulares, el programa realiza una verificación adicional comparando la cadena ingresada con un arreglo que contiene las palabras reservadas del lenguaje Java. Esta comprobación garantiza que el identificador no coincida con ninguna palabra reservada, condición necesaria para considerarlo válido dentro del lenguaje.

Finalmente, el resultado del análisis es mostrado al usuario mediante un mensaje emergente, indicando si la cadena ingresada cumple o no con los criterios establecidos. Los resultados obtenidos permiten observar que el uso de expresiones regulares facilita el análisis de cadenas de símbolos y constituye una alternativa eficiente y clara para la validación de lenguajes regulares.

Adicionalmente, el código fuente se encuentra disponible en el repositorio de GitHub: Detector de Cadenas de Caracteres con Gramática Formal

6. Resultados

En esta sección se presentan los resultados obtenidos a partir de la ejecución de los programas desarrollados durante la práctica. Las pruebas realizadas permitieron verificar el correcto funcionamiento de los algoritmos de análisis de cadenas de símbolos, tanto mediante estructuras de control y análisis carácter por carácter, como a través del uso de expresiones regulares.

Resultados del análisis mediante estructuras de datos

El primer programa permitió identificar correctamente los distintos tipos de palabras solicitados, tales como números enteros, palabras en minúsculas, palabras en mayúsculas e identificadores. A partir de las cadenas ingresadas por el usuario, el sistema analizó cada carácter individualmente, verificando que se encontrara dentro del rango permitido para cada categoría.

Durante las pruebas se observó que las cadenas que contenían caracteres fuera del rango establecido eran clasificadas correctamente como inválidas. En el caso de los identificadores, la combinación del uso de las funciones `isJavaIdentifierStart` y `isJavaIdentifierPart`, junto con la verificación contra el arreglo de palabras reservadas, permitió distinguir adecuadamente entre identificadores válidos e inválidos.

Las siguientes imagenes muestran la ejecución del programa:

La Figura 1 se observa el menu de opción del programa.

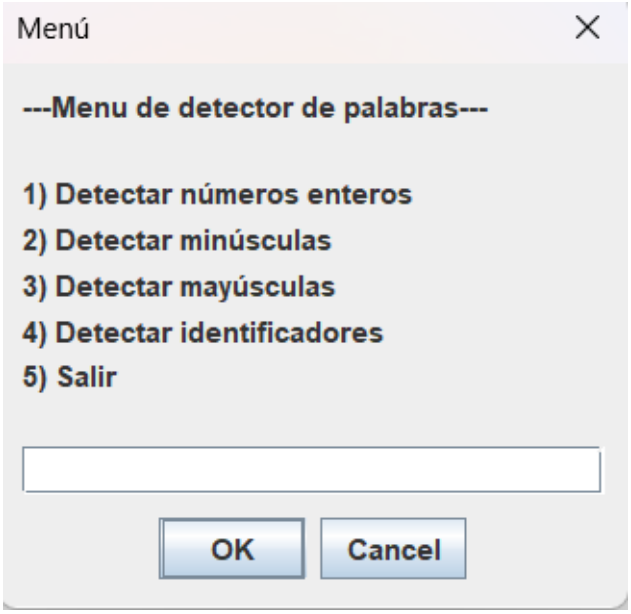

Figure 1: Menú detector de cadenas de caracteres

En la Figura 2 se observa el ingreso de la cadena de caracteres que se busque analizar.

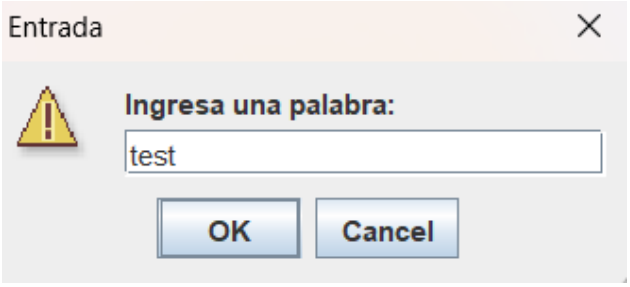

Figure 2: Ingreso de cadena de caracteres

En la Figura 3 se observa la salida ó resultado dado por el programa, en este caso se determina que la palabra ingresada si se conforma unicamente de minúsculas

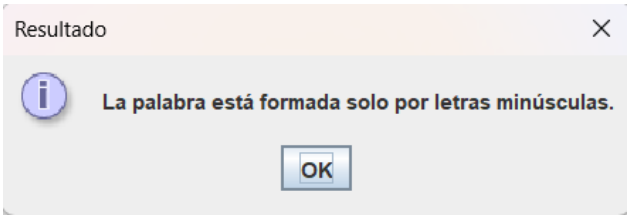

Figure 3: Resultado del programa

Resultados del análisis mediante expresiones regulares

En la segunda parte de la práctica se definieron expresiones regulares para cada tipo de palabra, incluyendo números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos utilizados en el lenguaje Java. Las pruebas realizadas demostraron que las expresiones regulares definidas permiten identificar de manera precisa las cadenas que cumplen con los patrones establecidos.

Los símbolos especiales fueron reconocidos correctamente cuando coincidían con el conjunto definido, mientras que cualquier carácter no contemplado en el alfabeto fue rechazado. En la Figura 4 se muestra un ejemplo de validación utilizando expresiones regulares, donde se observa la correcta clasificación de la cadena ingresada.

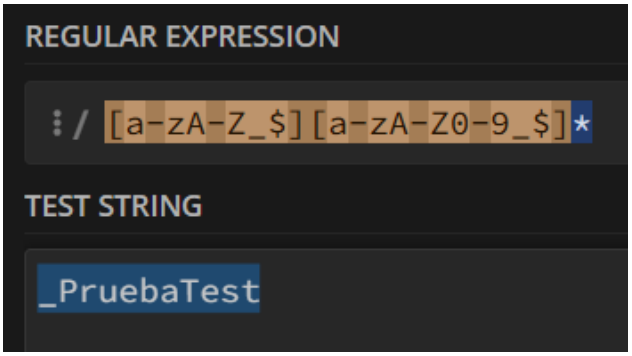

Figure 4: Prueba de identificadores en regex101

Resultados del programa con validación mediante expresiones regulares

El tercer programa integró las expresiones regulares directamente en el código, utilizando el método `matches()` para evaluar la cadena completa ingresada por el usuario. Los resultados obtenidos indican que este enfoque permite una validación más clara y concisa, reduciendo la cantidad de instrucciones necesarias en comparación con el análisis carácter por carácter.

Al igual que en el primer programa, en el caso de los identificadores se realizó una validación adicional contra el arreglo de palabras reservadas del lenguaje Java, lo cual evitó que palabras reservadas fueran aceptadas como nombres de variables. Los mensajes mostrados al usuario reflejaron correctamente si la cadena cumplía o no con los criterios establecidos. La Figura 5 y la Figura 6 presenta un ejemplo del resultado obtenido durante la ejecución del programa.

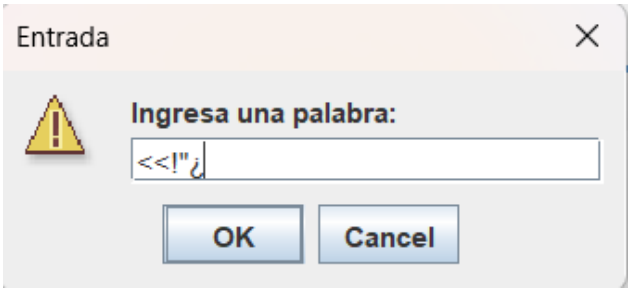

Figure 5: Ingreso de cadena de caracteres

En este caso la Figura 6 muestra el resultado correcto ya que el símbolo ¿ no es uno utilizado en la programación en lenguaje Java.

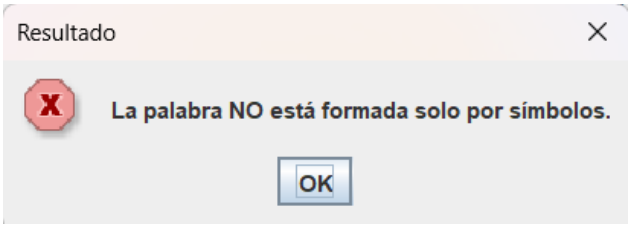

Figure 6: Salida del programa

En general, los resultados obtenidos demuestran que ambos enfoques permiten analizar cadenas de símbolos de manera correcta; sin embargo, el uso de expresiones regulares facilita la implementación y mejora la legibilidad del código, especialmente cuando se trabaja con lenguajes regulares bien definidos.

7. Conclusiones

A partir del desarrollo de la práctica fue posible comprender el proceso de análisis de cadenas de símbolos pertenecientes a un lenguaje regular, así como la importancia de definir correctamente un alfabeto y los criterios de validación para cada tipo de palabra. La implementación de los programas permitió reforzar conceptos fundamentales relacionados con gramáticas formales y lenguajes regulares.

El análisis carácter por carácter realizado en el primer programa demostró ser una estrategia efectiva para identificar errores específicos dentro de una cadena; sin embargo, también evidenció una mayor complejidad en la implementación, especialmente cuando se incrementa el número de reglas de validación. Este enfoque permitió entender de manera más detallada cómo se evalúan individualmente los símbolos de una palabra.

Por otro lado, el uso de expresiones regulares facilitó considerablemente el proceso de validación, ya que permitió evaluar cadenas completas de forma más clara y concisa. La definición de patrones mediante expresiones regulares resultó ser una alternativa eficiente para el análisis de lenguajes regulares, reduciendo la cantidad de código y mejorando su legibilidad.

La validación de identificadores mostró la necesidad de complementar las expresiones regulares con verificaciones adicionales, como la comparación contra palabras reservadas del lenguaje Java, lo cual permitió asegurar una clasificación más precisa. Esto evidenció que, en algunos casos, el uso combinado de diferentes técnicas de análisis produce resultados más robustos.

En conclusión, la práctica permitió comparar distintos métodos de análisis de cadenas de símbolos y reconocer las ventajas del uso de expresiones regulares en el desarrollo de programas. Los conocimientos adquiridos resultan relevantes para aplicaciones posteriores relacionadas con el análisis léxico, el diseño de compiladores y el procesamiento de lenguajes formales.

Referencias Bibliográficas

References

[1] Hopcroft, J. E.; Motwani, R.; Ullman, J. D. (**2006**). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson.

[2] Sipser, M. (**2012**). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.

[3] Chomsky, N. (**1959**). On certain formal properties of grammars. *Information and Control*, *2*(2), 137–167.

[4] Aho, A. V.; Lam, M. S.; Sethi, R.; Ullman, J. D. (**2007**). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.

[5] Friedl, J. E. F. (**2006**). *Mastering regular expressions* (3rd ed.). O'Reilly Media.

[6] Russell, S.; Norvig, P. (**2017**). *Artificial intelligence: A modern approach* (3rd ed.). Pearson.

[7] Sebesta, R. W. (**2016**). *Concepts of programming languages* (11th ed.). Pearson.

[8] Aho, A. V.; Lam, M. S.; Sethi, R.; Ullman, J. D. (**2007**). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.